NEXUS CPU ANALYZER

Complete Technology Stack & Architecture Guide

Prepared on: 13 November 2026, 06:04 PM

This document covers every technology, library, design pattern, and architectural decision used to build the NEXUS Real-Time CPU Analyzer — a professional-grade system monitoring dashboard built with Python, Flask, and Vanilla JavaScript. Use this as your complete reference for interviews, presentations, or project documentation.

1. Project Overview

Property	Details
Project Name	NEXUS CPU Analyzer (NEXUS PRO)
Type	Real-Time System Monitoring Dashboard (Web App)
Platform	Windows 10/11 — runs locally on your machine
Server Port	http://localhost:8050
Language	Python 3.11+ (Backend) | Vanilla JS ES6+ (Frontend)
Architecture	Flask SPA + REST API + Daemon Thread + Deque Buffer
Key Features	Live CPU/RAM/Network charts, Process Manager, Smart Countermeasures, Hardware Digital Twin, Z-Score Anomaly Detection, Multi-format Export

2. Backend Stack (Python)

2a. Flask — Web Framework

Aspect	Detail
What is it?	Lightweight Python web framework — turns Python functions into HTTP endpoints
Version	Flask 3.x
Role in NEXUS	Acts as the web server — serves the HTML dashboard and all /api/* REST routes
Key feature	Threaded mode (threaded=True) — handles multiple browser requests simultaneously
Why Flask?	Minimal, fast, no bloat. Dash framework was too heavy — caused 6s load & silent crashes
Key imports	Flask, jsonify, render_template, request, Response, send_file

2b. psutil — System Telemetry Engine

Function Called	What it Returns
psutil.cpu_percent(percpu=True)	Per-core CPU usage list (e.g. [45.2, 12.0, 78.5 …])

Function Called	What it Returns
psutil.virtual_memory()	RAM: total, used, available, percent
psutil.swap_memory()	Swap/page file: total, used, percent
psutil.net_io_counters()	Network: bytes_sent, bytes_recv (cumulative)
psutil.disk_partitions()	All mounted drives with filesystem type
psutil.disk_usage(path)	Per-disk: total, used, free, percent
psutil.process_iter(attrs)	All running processes with PID, name, CPU, RAM, status, user
psutil.cpu_freq()	CPU clock speed — current and max MHz
psutil.cpu_count(logical=False)	Physical core count (None-safe with 'or 1' fallback)
psutil.boot_time()	System boot timestamp → used to calculate uptime
NOTE	psutil.sensors_temperatures() does NOT exist on Windows — fully removed!

2c. ReportLab — PDF Generation

Module	Used For
SimpleDocTemplate	Creates the PDF document with page size, margins
Paragraph + ParagraphStyle	Styled text blocks — fonts, colours, alignment, spacing
Table + TableStyle	Data tables with alternating row colours, borders, header styling
HRFlowable	Horizontal divider lines between sections
colors.HexColor()	Custom hex colour codes (matches NEXUS brand colours)
io.BytesIO	Builds PDF in memory (RAM) — no temp file needed on disk
Used in	/api/export-pdf-report AND generate_interview_pdf.py

2d. Python Standard Library Modules

Module	Role in NEXUS
threading	Creates the background daemon thread that samples CPU data every 1 second
threading.Lock()	data_lock + proc_lock — prevents race conditions between threads
collections.deque	Rolling circular buffer — stores last 300 samples, auto-purges old data
statistics	mean(), stdev() — used for Z-score anomaly detection algorithm
csv + io.StringIO	CSV export — builds CSV string in memory, streams to browser as download
json	JSON export endpoint + all API responses (jsonify wraps this)
datetime + time	Timestamps, uptime calculation, export filename generation
os	File paths, PID safety check (os.getpid, os.getppid), env detection
platform	System info — OS name, version, machine architecture

3. Frontend Stack (Browser)

3a. HTML5 — Structure

Single Page Application (SPA) pattern — all 9 tabs are always present in the DOM. Switching tabs = toggling CSS `display:none / display:block`. Zero page reloads. Zero flicker. Instant tab switch (0ms). Semantic HTML5 elements: <header>, <nav>, <main>, <section>, <footer>.

3b. CSS3 — Glassmorphism Dark Theme (~1,300 lines)

CSS Feature	Used For
CSS Custom Properties	--purple, --blue, --panel-bg etc. — single-source theming
CSS Grid	2-column and 3-column responsive panel layouts
Flexbox	Navbar, tab bar, button groups, stat rows
backdrop-filter: blur	Glassmorphism frosted glass panel effect
CSS Keyframe Animations	Fade-in, pulse (live indicator), bar fill animations
CSS Transitions	Hover effects on buttons, cards, tabs (0.2s ease)
Media Queries	Responsive layout — collapses to 1-column on small screens
CSS Gradient	Background mesh gradient + card header gradients
:root variables	Theme colour palette — easy to change entire look in one place

3c. Vanilla JavaScript ES6+ (~1,050 lines)

JS Feature / API	Role in NEXUS
fetch() API	Calls Flask REST endpoints every 1 second — async, non-blocking
setInterval()	Master polling loop — drives all real-time updates
async / await	Clean async fetch without callback hell
DOM manipulation	document.getElementById, innerHTML, textContent — updates all values live
Event Listeners	Tab clicks, button clicks, modal open/close, form submit, keyboard
window.addEventListener('resize')	Triggers chart.resize() — keeps charts responsive
ES6 Arrow functions	() => {} — concise callbacks throughout
Template literals	Dynamic HTML building with ${variable} interpolation
Destructuring	const { cpu_percent, memory } = data — clean data unpacking
Optional chaining	data?.memory?.percent — prevents crashes if API field missing
Nullish coalescing	value ?? 0 — safe fallback for missing values

3d. Chart.js 4.4.2 — Real-Time Charts

Aspect	Detail
What is it?	Open-source JavaScript charting library — canvas-based, very fast

Aspect	Detail
Version	4.4.2 (bundled locally as chart.min.js — 205KB, works fully offline)
Chart Types Used	Line charts — CPU history, Memory history, Network sent/received
Real-time update	chart.data.labels.push() + chart.data.datasets[0].data.push() + chart.update()
Max data points	60 points per chart — older points shift out (FIFO rolling window)
Why local bundle?	No CDN dependency — dashboard works without internet connection
Responsive	chart.resize() called on window resize event — always fits panel width
Animations	Disabled (animation: false) for real-time performance — no lag on updates

4. Architecture & Design Patterns

4a. Full System Data Flow

Here is how data flows from your CPU hardware all the way to the browser chart:

Step	Layer	What Happens
1	OS Kernel	Windows kernel tracks CPU usage, process list, network bytes
2	psutil	Python library reads kernel data via Windows API calls
3	Daemon Thread	Background thread calls psutil every 1 second, stores in deque
4	deque buffer	Circular buffer holds last 300 samples (5 minutes of history)
5	Flask API route	Browser requests /api/metrics → Flask reads deque → returns JSON
6	fetch() in JS	Browser's fetch() receives JSON response
7	DOM update	JS updates all number displays, gauge fills, table rows
8	Chart.js	JS pushes new data point to chart → chart.update() redraws line
9	User sees	Live charts, live numbers, live process list — updated every 1s

4b. Design Patterns Used

Pattern	Where Used	Why
Single Page App (SPA)	index.html — 9 tabs	Zero page reloads, instant tab switch
Producer-Consumer	Thread → deque → API	Decouples data collection from serving
Circular Buffer	deque(maxlen=300)	Fixed memory — never grows unbounded
Thread-safe Locking	data_lock, proc_lock	Prevents race conditions / data corruption
REST API	All /api/* routes	Clean separation of frontend and backend
Z-Score Detection	Anomaly detection engine	(value−mean)/stdev > 2.6 → spike alert
Singleton Settings	Global settings dict	Shared config across all API routes
Guard Clause	Kill/terminate endpoints	BSOD + self-kill protection
In-memory streaming	CSV/JSON/PDF export	io.StringIO / io.BytesIO — no temp files

5. All REST API Endpoints

Method	Endpoint	Returns / Does
GET	/	Serves the full HTML dashboard (index.html)
GET	/api/metrics	CPU%, per-core, RAM, swap, network, uptime, kernel stats
GET	/api/processes	Process list — filterable by search, sortable, paginated
POST	/api/kill/<pid>	Terminates a process (with safety checks)
GET	/api/countermeasures	Rogue process analysis + reclaimable RAM/CPU stats
POST	/api/countermeasures/terminate/<pid>	User-authorized countermeasure kill
POST	/api/countermeasures/boost	Batch turbo boost — kills all rogue hogs at once
GET	/api/hardware-twin	Full hardware digital twin JSON snapshot
GET	/api/disks	All disk partitions with usage stats
GET	/api/alerts	Smart alert log (filterable by level)
POST	/api/alerts/clear	Clears all alerts from the log
GET	/api/anomalies	Z-score detected CPU spike anomalies
GET	/api/system-info	Host system info — OS, Python, hostname, IP
GET	/api/settings	Read current threshold/refresh settings
POST	/api/settings	Save updated settings (warn%, crit%, refresh rate)
GET	/api/export-csv	Download session history as CSV (Excel-ready)
GET	/api/export-json	Download full monitoring dump as JSON
GET	/api/export-pdf-report	Download live snapshot as professional PDF

6. Project File Structure

File / Folder	Size (approx)	Purpose
app.py	~750 lines	Flask backend — all routes, daemon thread, psutil engine, export logic
templates/index.html	~900 lines	Full SPA — 9 tabs, all UI panels, modals, navbar
static/style.css	~1,300 lines	Glassmorphism dark theme — all CSS variables, animations, responsive
static/script.js	~1,050 lines	Vanilla JS engine — polling, chart updates, tab switching, modals
static/chart.min.js	~205 KB	Chart.js 4.4.2 — bundled locally, no CDN, works offline
generate_interview_pdf.py	~200 lines	Personal interview guide PDF generator (not exposed on website)
generate_techstack_pdf.py	~300 lines	This file — generates NEXUS_TechStack_Guide.pdf
NEXUS_CPU_Analyzer_Interview_Guide.pdf	~15 KB	Personal interview prep PDF (personal use only)
NEXUS_TechStack_Guide.pdf	~30 KB	This output — tech stack documentation PDF
START_NEXUS.bat	~10 lines	One-click Windows launcher — double-click to start server

7. Safety Systems & Windows Compatibility

Safety Feature	Implementation	Why It Matters
BSOD Guard	PROTECTED_PROCESSES set	Blocks kill of csrss.exe, lsass.exe, wininit.exe, services.exe etc.

Safety Feature	Implementation	Why It Matters
Self-kill Prevention	os.getpid() / os.getppid()	Server cannot accidentally kill itself via process manager
Windows API Compatibility	sensors_temperatures() removed	This psutil function crashes on Windows — removed entirely
None-safe cpu_count	psutil.cpu_count() or 1	Returns None in VM environments — fallback prevents division by zero
Thread-safe reads	threading.Lock() on deque	Prevents data corruption when thread writes while API reads
Memory-safe buffer	deque(maxlen=300)	Hard cap on history — no RAM leak during long sessions
In-memory exports	io.BytesIO / io.StringIO	No temp files on disk — cleaner, faster, no permission issues
Error handling	try/except on all psutil calls	Single failing sensor doesn't crash the entire API response

8. Key Architecture Decisions

Decision	Chosen	Rejected	Reason
Web framework	Flask	Plotly Dash	Dash caused 6s load, blank charts, silent callback crashes
Frontend framework	Vanilla JS	React / Vue	Zero build tools, zero npm, instant load, easier to debug
Chart library	Chart.js (local)	Plotly.js (CDN)	No internet dependency, 205KB vs 3MB, faster
Data storage	deque in RAM	SQLite / file	No disk I/O, microsecond access, auto-purge built-in
Concurrency model	1 daemon thread	asyncio	psutil is blocking — threads suit it better than async
Tab switching	CSS display toggle	Router / AJAX	0ms switch, all data already in DOM, no refetch needed
PDF generation	ReportLab	WeasyPrint	ReportLab is pure Python, no headless browser needed
Export file creation	In-memory buffer	Temp file on disk	Cleaner, no cleanup needed, no OS permission issues